

# React & Next.js

Jour 1 — Fondamentaux



# Sommaire — Jour 1

---

**01** Révisions ES6+ — variables & scope

**03** Arrow functions & closures

**05** Modules & async/await

**07** Philosophie & Virtual DOM

**09** Initialiser un projet React (Vite)

**02** Prog. déclarative vs impérative

**04** Spread, rest & destructuration

**06** Limites de JavaScript vanilla

**08** JSX — HTML dans JavaScript

**10** TP 01 — Premier projet React



# Programme du Jour

---

- **Matin (9h–12h30)** : Révisions ES6+ intensives avec exercices live
- **Déjeuner (12h30–13h30)**
- **Après-midi (13h30–17h30)** : Introduction à React + TP guidé
- **Fin de journée** : Quiz de validation + Q&A

💡 *Toutes les corrections des exercices sont disponibles sur le dépôt GitHub de la formation*

# | 01 · Révisions ES6+



## Pourquoi ES6+ en 2025 ?

---

- **React** est écrit en ES6+ et repose sur ses fonctionnalités natives
- Les **hooks**, **JSX compilé** et les **patterns** React utilisent massivement :
  - Destructuration, spread, arrow functions
  - Modules, async/await, méthodes fonctionnelles
- Comprendre ES6+ = comprendre React **en profondeur**

*Sans ES6+, React reste une boîte noire. Avec ES6+, il devient transparent.*

**var**

# — Le problème historique

```
// Problème 1 : function scope (pas de block scope)
if (true) {
  var message = "Bonjour";
}
console.log(message); // "Bonjour" ← accessible en dehors du bloc !

// Problème 2 : hoisting
console.log(x); // undefined (pas d'erreur !)
var x = 42;

// Problème 3 : re-déclaration silencieuse
var count = 1;
var count = 2; // aucune erreur... danger !
```

- **var** est **hissé** (hoisted) en haut de sa fonction
- Portée **fonction** et non **bloc** → source de bugs difficiles à tracer



## let — Le scope de bloc

```
// Block scope : la variable vit dans son bloc {}
if (true) {
  let message = "Bonjour";
  console.log(message); // "Bonjour" ✓
}
console.log(message); // ReferenceError ✓ (comportement attendu)

// Pas de re-déclaration
let count = 1;
let count = 2; // SyntaxError ✓

// Temporal Dead Zone (TDZ)
console.log(x); // ReferenceError ✓
let x = 42;

// Idéal pour les boucles
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
  // Affiche 0, 1, 2 (et non 3, 3, 3 avec var)
}
```

**const**

# — L'immutabilité du binding

```
// const = binding immuable (pas la valeur !)  
const PI = 3.14159;  
PI = 3; // TypeError ✓  
  
// ⚠ Avec les objets : le contenu reste mutable  
const user = { name: "Alice", age: 30 };  
user.age = 31; // OK : on mute la propriété  
user = { name: "Bob" }; // TypeError : on ne peut pas réassigner  
  
// ⚠ Avec les tableaux : même logique  
const fruits = ["pomme", "poire"];  
fruits.push("banane"); // OK ✓  
fruits = ["kiwi"]; // TypeError ✗  
  
// En React : toujours préférer const pour les composants  
const MyComponent = () => { ... };
```





# Récapitulatif

`var``/``let``/``const`

| Caractéristique | <code>var</code> | <code>let</code> | <code>const</code> |
|-----------------|------------------|------------------|--------------------|
| Portée          | Fonction         | Bloc             | Bloc               |
| Hoisting        | ✓ (undefined)    | ✓ (TDZ)          | ✓ (TDZ)            |
| Re-déclaration  | ✓                | ✗                | ✗                  |
| Réassignation   | ✓                | ✓                | ✗                  |
| Objet mutable   | ✓                | ✓                | ✓                  |
| Usage React     | ✗ Éviter         | ⚠ Rare           | ✓ Par défaut       |

**Règle d'or React :** Toujours commencer par `const`. Passer à `let` uniquement si la réassignation est nécessaire. Ne jamais utiliser `var`.

## | 02 · Programmation Déclarative

# Impératif vs Déclaratif

## Impératif

*Comment faire ?*

```
// Filtrer les nombres pairs
const numbers = [1,2,3,4,5,6];
const evens = [];

for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] % 2 === 0) {
    evens.push(numbers[i]);
  }
}
```

- Décrit **chaque étape**
- Mutabilité explicite
- Difficile à composer

## Déclaratif

*Quoi obtenir ?*

```
// Même résultat, style déclaratif
const numbers = [1,2,3,4,5,6];
const evens = numbers.filter(
  n => n % 2 === 0
);
```

- Décrit **l'intention**
- Pas de mutation
- Composable et lisible

# Déclaratif — En pratique avec React

```
// ❌ Impératif : manipulation directe du DOM
document.getElementById('list').innerHTML = '';
data.forEach(item => {
  const li = document.createElement('li');
  li.textContent = item.name;
  document.getElementById('list').appendChild(li);
});

// ✅ Déclaratif : décrire ce qu'on veut afficher
function ItemList({ data }) {
  return (
    <ul>
      {data.map(item => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
}
```

React gère **comment** mettre à jour le DOM. Vous décrivez **quoi** afficher.

## | 03 · Arrow Functions & Closures

## Arrow Functions — Syntaxes

---

```
// Fonction classique
function add(a, b) { return a + b; }

// Arrow function – version complète
const add = (a, b) => { return a + b; };

// Arrow function – retour implicite (1 expression)
const add = (a, b) => a + b;

// 1 seul paramètre → parenthèses optionnelles
const double = x => x * 2;

// Retourner un objet littéral : wrapper avec ()
const toObject = (name) => ({ name: name });
const toObject = (name) => ({ name }); // shorthand

// Sans paramètre
const greet = () => "Hello !";
```

## ➡ Arrow Functions — `this` binding

```
// ❌ Problème avec fonction classique
const timer = {
  seconds: 0,
  start: function() {
    setInterval(function() {
      this.seconds++; // ⚠️ 'this' = window/undefined !
      console.log(this.seconds); // NaN
    }, 1000);
  }
};

// ✅ Solution : arrow function (hérite du 'this' parent)
const timer = {
  seconds: 0,
  start: function() {
    setInterval(() => {
      this.seconds++; // ✅ 'this' = timer
      console.log(this.seconds); // 1, 2, 3...
    }, 1000);
  }
};
```

## Arrow Functions — Usage React

---

```
// Callbacks dans map/filter/reduce
const doubled = [1, 2, 3].map(n => n * 2); // [2, 4, 6]

// Gestionnaires d'événements (pas de problème de this)
function Button() {
  const handleClick = () => {
    console.log("Cliqué !");
  };
  return <button onClick={handleClick}>Cliquer</button>;
}

// Inline dans JSX
function List({ items }) {
  return (
    <ul>
      {items.map(item => <li key={item.id}>{item.name}</li>)}
    </ul>
  );
}
```



## | 04 · Spread, Rest & Destructuration

# Spread Operator — Tableaux

```
// Copier un tableau (shallow copy)
const original = [1, 2, 3];
const copy = [...original]; // [1, 2, 3]

// Fusionner des tableaux
const a = [1, 2];
const b = [3, 4];
const merged = [...a, ...b]; // [1, 2, 3, 4]

// Ajouter un élément (immutable)
const fruits = ["pomme", "poire"];
const newFruits = [...fruits, "banane"]; // sans muter fruits

// En React : mise à jour de state immuable
const [items, setItems] = useState([1, 2, 3]);
const addItem = (item) => setItems([...items, item]);
```

## Spread Operator — Objets

```
// Copier un objet (shallow copy)
const user = { name: "Alice", age: 30 };
const copy = { ...user }; // { name: "Alice", age: 30 }

// Fusionner des objets
const defaults = { theme: "light", lang: "fr" };
const custom   = { lang: "en", fontSize: 14 };
const config   = { ...defaults, ...custom };
// { theme: "light", lang: "en", fontSize: 14 }
// ↑ custom écrase defaults pour les clés communes

// En React : mettre à jour un objet dans le state
const [user, setUser] = useState({ name: "Alice", age: 30 });
const updateAge = () => setUser({ ...user, age: 31 });
// ✓ On ne mute pas l'objet original
```

# Rest Parameters

---

```
// Rest : regrouper le reste des arguments
function sum(...numbers) {
  return numbers.reduce((acc, n) => acc + n, 0);
}
sum(1, 2, 3, 4); // 10

// Combiné avec des arguments nommés
function log(level, ...messages) {
  console.log(`[${level}]`, ...messages);
}
log("INFO", "Serveur", "démarré"); // [INFO] Serveur démarré

// En destructuration : capturer le reste
const [first, second, ...rest] = [1, 2, 3, 4, 5];
// first = 1, second = 2, rest = [3, 4, 5]

const { name, ...otherProps } = { name: "Alice", age: 30, city: "Paris" };
// name = "Alice", otherProps = { age: 30, city: "Paris" }
```

# Destructuration — Tableaux

```
// Syntax de base
const [x, y, z] = [10, 20, 30];
console.log(x); // 10

// Ignorer des valeurs
const [first, , third] = [1, 2, 3];
console.log(third); // 3

// Valeurs par défaut
const [a = 0, b = 0] = [5];
console.log(a, b); // 5, 0

// Swap de variables
let p = 1, q = 2;
[p, q] = [q, p];
console.log(p, q); // 2, 1

// Pattern React : useState retourne un tableau
const [count, setCount] = useState(0);
```

# Destructuration — Objets

```
// Syntaxe de base
const { name, age } = { name: "Alice", age: 30, city: "Paris" };
console.log(name, age); // "Alice" 30

// Renommage
const { name: userName, age: userAge } = user;
console.log(userName); // "Alice"

// Valeurs par défaut
const { theme = "light", lang = "fr" } = config;

// Imbriquée
const { address: { city, zip } } = {
  address: { city: "Paris", zip: "75001" }
};

// Dans les paramètres de fonction (très courant en React)
function UserCard({ name, age, avatar = "/default.png" }) {
  return <div>{name} - {age}</div>;
}
```



# Template Literals

```
const name = "Alice";
const age = 30;

// ✗ Concaténation classique
const msg1 = "Bonjour " + name + ", vous avez " + age + " ans.";

// ✔ Template literal
const msg2 = `Bonjour ${name}, vous avez ${age} ans.`;

// Expressions arbitraires
const price = 19.99;
const tva = `Prix TTC : ${price * 1.2}.toFixed(2)} €`;

// Multi-lignes
const html = `
  <div class="card">
    <h2>${name}</h2>
    <p>Âge : ${age}</p>
  </div>
`;

// En React (className dynamiques)
const className = `btn ${isActive ? 'btn-active' : 'btn-inactive'}`;
```

## | 05 · Modules ES6 & Async/Await



# Modules ES6 — Export

---

```
// — math.js —————

// Named exports (plusieurs par fichier)
export const PI = 3.14159;
export function add(a, b) { return a + b; }
export const multiply = (a, b) => a * b;

// Ou groupé à la fin
const subtract = (a, b) => a - b;
const divide = (a, b) => a / b;
export { subtract, divide };

// Default export (un seul par fichier)
export default function Calculator() { ... }

// — En React : chaque composant dans son fichier —
// Button.jsx
export default function Button({ label, onClick }) {
  return <button onClick={onClick}>{label}</button>;
}
```



# Modules ES6 — Import

```
// Importer les named exports
import { PI, add, multiply } from './math.js';
console.log(PI);           // 3.14159
console.log(add(2,3)); // 5

// Renommer à l'import
import { add as sum } from './math.js';
sum(1, 2); // 3

// Importer tout
import * as Math from './math.js';
Math.add(1, 2); // 3

// Importer le default export
import Calculator from './math.js'; // Nom libre

// Combiner default + named
import Button, { ButtonGroup } from './Button.jsx';

// En React : imports du quotidien
import React, { useState, useEffect } from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
```



# Promises — Concept

```
// Une Promise représente une valeur future (asynchrone)
// États : pending → fulfilled | rejected

const fetchUser = (id) => {
  return new Promise((resolve, reject) => {
    if (id > 0) {
      resolve({ id, name: "Alice" }); // Succès
    } else {
      reject(new Error("ID invalide")); // Échec
    }
  });
};

// Consommer une Promise avec .then() / .catch()
fetchUser(1)
  .then(user => console.log(user.name)) // "Alice"
  .catch(err => console.error(err.message))
  .finally(() => console.log("Terminé"));
```



## async / await — La syntaxe moderne

```
// async/await : sucre syntaxique sur les Promises
// Rend le code asynchrone lisible comme du code synchrone

// ✅ Version async/await
async function getUser(id) {
  try {
    const response = await fetch(`/api/users/${id}`);

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }

    const user = await response.json();
    return user;
  } catch (error) {
    console.error("Erreur :", error.message);
    throw error; // Re-throw pour que l'appelant gère l'erreur
  }
}

// Appel
const user = await getUser(1);
console.log(user.name);
```



## async / await — Patterns avancés

```
// Requêtes parallèles avec Promise.all
async function fetchDashboard(userId) {
  const [user, posts, comments] = await Promise.all([
    fetch(`/api/users/${userId}`).then(r => r.json()),
    fetch(`/api/posts?userId=${userId}`).then(r => r.json()),
    fetch(`/api/comments?userId=${userId}`).then(r => r.json()),
  ]);
  return { user, posts, comments };
}

// En React : pattern classique dans useEffect
useEffect(() => {
  async function loadData() {
    try {
      const data = await fetchDashboard(userId);
      setDashboard(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }
  loadData();
}, [userId]);
```

# Optional Chaining & Nullish Coalescing

```
// Optional Chaining (?.) – accès sécurisé aux propriétés
const user = null;

// ❌ Ancienne façon
const city = user && user.address && user.address.city;

// ✅ Optional chaining
const city = user?.address?.city; // undefined (pas d'erreur)
const zip = user?.address?.zip ?? "00000"; // Avec valeur par défaut

// Avec méthodes
const length = user?.getName?.(); // undefined si getName n'existe pas

// Nullish Coalescing (??) – valeur par défaut si null/undefined
const count = null ?? 0; // 0
const count = undefined ?? 0; // 0
const count = 0 ?? 42; // 0 (0 n'est pas null/undefined !)
const count = "" ?? "défaut"; // "" (chaîne vide préservée)

// Différence avec || (qui court-circuite sur les falsy)
const count = 0 || 42; // 42 ← attention !
```



# Méthodes Fonctionnelles —

map

```
const users = [
  { id: 1, name: "Alice", age: 28 },
  { id: 2, name: "Bob", age: 35 },
  { id: 3, name: "Carol", age: 24 },
];

// map : transformer chaque élément → nouveau tableau de même taille
const names = users.map(u => u.name);
// ["Alice", "Bob", "Carol"]

const withFullInfo = users.map(u => ({
  ...u,
  label: `${u.name} (${u.age} ans)`,
}));

// En React : rendu de listes
function UserList({ users }) {
  return (
    <ul>
      {users.map(u => (
        <li key={u.id}>{u.name}</li>
      ))}
    </ul>
  );
}
```



# Méthodes Fonctionnelles — `filter` & `reduce`

```
const users = [
  { id: 1, name: "Alice", age: 28, active: true },
  { id: 2, name: "Bob", age: 35, active: false },
  { id: 3, name: "Carol", age: 24, active: true },
];

// filter : garder seulement les éléments qui passent le test
const activeUsers = users.filter(u => u.active);
// [Alice, Carol]

// reduce : agréger en une seule valeur
const totalAge = users.reduce((acc, u) => acc + u.age, 0); // 87
const byId = users.reduce((acc, u) => {
  acc[u.id] = u;
  return acc;
}, {}); // { 1: Alice, 2: Bob, 3: Carol }

// Chaîner les méthodes (pattern React courant)
const activeNames = users
  .filter(u => u.active)
  .map(u => u.name.toUpperCase());
// ["ALICE", "CAROL"]
```





# Récapitulatif ES6+

## Variables

`const` par défaut, `let` si réassignation nécessaire, jamais `var`

## Arrow Functions

Syntaxe concise + héritage du `this` parent — indispensable en React

## Spread & Destructuration

Copie immuable des objets/tableaux, paramètres lisibles dans les composants

## Async/Await

Gestion propre de l'asynchronisme — base des appels API dans

`useEffect`

## | 06 · Les Limites de JavaScript Vanilla



# Le problème de la manipulation DOM

```
// Application sans framework : état dans le DOM
function updateUI(users) {
  const list = document.getElementById('user-list');
  list.innerHTML = ''; // Supprimer tout

  users.forEach(user => {
    const li = document.createElement('li');
    li.className = user.active ? 'active' : '';
    li.setAttribute('data-id', user.id);
    li.textContent = user.name;
    li.addEventListener('click', () => deleteUser(user.id));
    list.appendChild(li); // Ajouter un par un
  });

  document.getElementById('count').textContent = users.length;
}
// → Lent, répétitif, et difficile à maintenir
// → Aucune réutilisabilité des éléments UI
```

# Le problème de la synchronisation

---

- **État dispersé** dans le DOM → source de vérité peu fiable
- **Synchronisation manuelle** entre les parties de l'UI
- **Event listeners** à gérer manuellement (fuites mémoire)
- **Pas de composants** → code dupliqué partout
- **Difficile à tester** → dépendance forte au DOM
- **Performance** → re-rendu complet inutile

*Imaginez une page e-commerce : panier, catalogue, stock, prix, filtres...  
Tout synchroniser manuellement devient un enfer.*

## | 07 · Philosophie & Virtual DOM



# Historique de React

---

- **2011** : Jordan Walke (Facebook) crée **FaxJS**, l'ancêtre de React
- **2013** : React open-sourcé à la conférence JSConf US
- **2015** : React Native → mobile (iOS & Android)
- **2016** : React 15 — adoption massive en entreprise
- **2019** : React Hooks → fin des class components
- **2022** : React 18 → **Concurrent Mode**, Suspense, transitions
- **2023** : React Server Components en production (Next.js 13+)
- **2024** : React 19 → Actions, Form hooks, `use()`, compiler



# Philosophie React

- **Composants** : l'UI est une arborescence de briques réutilisables
- **Unidirectional Data Flow** : les données descendent, les events remontent
- **Déclaratif** : décrivez *quoi* afficher, pas *comment* manipuler le DOM
- **Immutabilité** : jamais muter le state directement → `setState()`

```
UI = f(state)
```

*L'interface est une **fonction pure** de l'état.*

*Même state → même affichage. Toujours prévisible.*

# Virtual DOM — Concept

---

## DOM Réel

- Objet C++ lourd du navigateur
- Chaque modification → **reflow + repaint**
- Mise à jour d'un nœud → potentiellement toute la page recalculée
- Coûteux pour des mises à jour fréquentes

## Virtual DOM

- **Objet JS** léger en mémoire
- Représentation de l'arbre DOM
- **Comparison (diff)** entre ancien et nouveau VDOM
- Seuls les **changements réels** sont appliqués au DOM





# Réconciliation React

State change



```
Nouveau Virtual DOM (tree JS)
{ type: 'ul', children: [
  { type: 'li', props: { key: 1 }, ... }
  { type: 'li', props: { key: 2 }, ... } ← NEW
]}
```



Algorithme de diff ( $O(n)$ )



Seul le nouveau `<li key=2>`  
est inséré dans le DOM réel

→ **Fiber** (React 16+) : réconciliation **incrémentale** et interruptible

## | 08 · JSX — HTML dans JavaScript

# abc JSX — Introduction

- **JSX** = JavaScript XML — extension de syntaxe pour JavaScript
- Permet d'écrire du HTML-like directement dans le code JS
- **N'est pas du HTML** : c'est du sucre syntaxique transformé par Babel/SWC

```
// Ce que vous écrivez (JSX)
const element = <h1 className="title">Bonjour React !</h1>;

// Ce que le compilateur produit (JS pur)
const element = React.createElement(
  'h1',
  { className: 'title' },
  'Bonjour React !'
);
```

*JSX est **optionnel** mais universellement utilisé dans l'écosystème React.*

```
// Fragment : retourner plusieurs éléments sans div wrapper
function App() {
  return (
    <>
      <h1>Titre</h1>
      <p>Paragraphe</p>
    </>
  );
}

// Toujours un seul élément racine (ou Fragment)
// ✗ Interdit :
return (
  <h1>Titre</h1>
  <p>Paragraphe</p> // Erreur : Adjacent JSX elements
);

// className, not class (class est un mot réservé JS)
<div className="container">...</div>

// htmlFor, not for
<label htmlFor="email">Email :</label>
```

# abc JSX — Expressions JavaScript

```
const name = "Alice";
const age = 30;
const isAdmin = true;

function Greeting() {
  return (
    <div>
      { /* Commentaire JSX */ }
      <h1>Bonjour, {name} !</h1>
      <p>Âge : {age} ans</p>
      <p>Rôle : {isAdmin ? "Admin" : "Utilisateur"}</p>
      <p>Score : {(Math.random() * 100).toFixed(1)}</p>
    </div>
  );
}

// ⚠ Valeurs ignorées (pas affichées)
// false, null, undefined, true → rendus comme "rien"
{false && <span>Caché</span>} // Rien affiché
{0 && <span>Attention !</span>} // Affiche "0" ! (falsy mais pas boolean)
```

```
function UserStatus({ user, isLoading }) {  
  // Pattern 1 : ternaire (recommandé pour 2 alternatives)  
  if (isLoading) {  
    return <p>Chargement...</p>;  
  }  
  
  return (  
    <div>  
      { /* Pattern 2 : && (afficher ou rien) */  
        {user.isAdmin && <span className="badge">Admin</span>}  
  
      { /* Pattern 3 : ternaire inline */  
        <p>Statut : {user.active ? "Actif" : "Inactif"}</p>  
  
      { /* Pattern 4 : variable JSX */  
        (() => {  
          if (user.score > 90) return <span>★ Expert</span>;  
          if (user.score > 50) return <span>👍 Intermédiaire</span>;  
          return <span>🌱 Débutant</span>;  
        })()  
      </div>  
    );  
  }  
}
```

## abc JSX — Rendu de listes

```
const fruits = [
  { id: 1, name: "Pomme",   emoji: "🍏" },
  { id: 2, name: "Banane",  emoji: "🍌" },
  { id: 3, name: "Cerise",  emoji: "🍒" },
];

function FruitList() {
  return (
    <ul>
      {fruits.map(fruit => (
        // ⚠️ key : obligatoire, unique parmi les frères, stable
        <li key={fruit.id}>
          {fruit.emoji} {fruit.name}
        </li>
      ))}
    </ul>
  );
}

// ❌ Ne pas utiliser l'index comme key si la liste peut être réordonnée
{items.map((item, index) => <li key={index}>...</li>)} // Dangereux !
```



# JSX — Attributs & Événements

```
function Form() {
  const handleSubmit = (e) => {
    e.preventDefault();
    console.log("Formulaire soumis");
  };

  return (
    <form onSubmit={handleSubmit}>
      /* Attributs : camelCase en JSX */
      <input
        type="text"
        className="input-field"
        placeholder="Votre nom"
        autoComplete="name"
        onChange={ (e) => console.log(e.target.value) }
      />
      /* Style inline : objet JS */
      <button
        type="submit"
        style={{ backgroundColor: "#0ea5e9", color: "white" }}
      >
        Envoyer
      </button>
    </form>
  );
}
```



## | 09 · Initialiser un Projet React

# Les Outils de Démarrage

---

## Create React App (CRA)

- Outil officiel **Meta** (2016)
- Configuration zero
- **Webpack** sous le capot
- Démarrage lent (>30s)
- Build lent
- ⚠ Plus maintenu activement

## Vite (recommandé)

- Développé par **Evan You** (Vue.js)
- **ES Modules natifs** en dev
- Démarrage < **1 seconde**
- **HMR** ultra-rapide
- Build avec **Rollup**
- Standard industrie 2024+

# Créer un Projet avec Vite

---

```
# Créer le projet (Node.js ≥ 18 requis)
npm create vite@latest mon-app -- --template react

# Ou avec le mode interactif
npm create vite@latest
# > Project name: mon-app
# > Framework: React
# > Variant: JavaScript (ou TypeScript)

# Installer les dépendances
cd mon-app
npm install

# Lancer le serveur de développement
npm run dev
# ✓ Local: http://localhost:5173
```

# Structure du Projet Vite + React

---

```
mon-app/
├── public/                # Fichiers statiques (servis tels quels)
│   └── vite.svg
├── src/                  # Code source
│   ├── assets/           # Images, fonts
│   ├── App.css            # Styles du composant App
│   ├── App.jsx            # ← Composant racine
│   ├── index.css          # Styles globaux
│   └── main.jsx           # ← Point d'entrée React
├── index.html             # HTML de base (Vite entry point)
├── package.json           # Dépendances & scripts
└── vite.config.js         # Configuration Vite
```



## package.json — Décryptage

```
{
  "name": "mon-app",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.0",
    "react-dom": "^18.3.0"
  },
  "devDependencies": {
    "@types/react": "^18.3.0",
    "@vitejs/plugin-react": "^4.3.0",
    "vite": "^5.4.0"
  }
}
```

**main.jsx**

# — Point d'entrée React

```
// src/main.jsx

import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

// Trouver le nœud DOM "racine" dans index.html
const root = document.getElementById('root')

// Créer la racine React et y monter le composant App
createRoot(root).render(
  <StrictMode>
    <App />
  </StrictMode>
)

// StrictMode :
// - Détecte les problèmes potentiels (dev uniquement)
// - Double les renders en dev pour détecter les effets de bord
// - N'a aucun impact en production
```

**App.jsx**

# — Composant Racine

```
// src/App.jsx

import { useState } from 'react'
import reactLogo from './assets/react.svg'
import './App.css'

function App() {
  const [count, setCount] = useState(0)

  return (
    <div className="app">
      <h1>Mon Application React</h1>
      <p>
        <button onClick={() => setCount(count + 1)}>
          Compteur : {count}
        </button>
      </p>
    </div>
  )
}

export default App
```



```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Mon App React</title>
  </head>
  <body>
    <!-- Le point de montage React -->
    <div id="root"></div>

    <!-- Vite injecte le script automatiquement -->
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```





# Premier Composant Fonctionnel

```
// src/components/Hello.jsx

// Un composant = une fonction qui retourne du JSX
function Hello({ name = "Monde" }) {
  return (
    <div className="hello-card">
      <h2>Bonjour, {name} ! 🙌</h2>
      <p>Mon premier composant React.</p>
    </div>
  );
}

export default Hello;

// Utilisation dans App.jsx
import Hello from './components/Hello';

function App() {
  return (
    <div>
      <Hello />           {/* name = "Monde" (défaut) */}
      <Hello name="Alice" />  {/* name = "Alice" */}
      <Hello name="Bob" />    {/* name = "Bob" */}
    </div>
  );
}
```



# Import / Export de Composants

```
// src/components/Button.jsx

// Named export
export function PrimaryButton({ label, onClick }) {
  return (
    <button className="btn-primary" onClick={onClick}>
      {label}
    </button>
  );
}

export function SecondaryButton({ label, onClick }) {
  return (
    <button className="btn-secondary" onClick={onClick}>
      {label}
    </button>
  );
}

// Dans App.jsx
import { PrimaryButton, SecondaryButton } from './components/Button';

// Convention : 1 fichier = 1 composant principal (default export)
// + possibilité de named exports pour des variantes
```

# L'Écosystème React 2025

---

## UI & Styling

Tailwind CSS, shadcn/ui, MUI, Radix UI, Ant Design

## Data Fetching

TanStack Query, SWR, Apollo (GraphQL), RTK Query

## State Management

Zustand, Redux Toolkit, Jotai, Valtio, Recoil

## Frameworks

Next.js, Remix, TanStack Start, Gatsby, Astro



# React vs Vue vs Angular

| Critère              | React           | Vue 3                | Angular            |
|----------------------|-----------------|----------------------|--------------------|
| Type                 | Bibliothèque UI | Framework progressif | Framework complet  |
| Créateur             | Meta (Facebook) | Evan You (2014)      | Google (2016)      |
| Langage              | JS / TS         | JS / TS              | TypeScript (natif) |
| Courbe apprentissage | Moyenne         | Douce                | Raide              |
| Popularité           | ★★★★★           | ★★★★                 | ★★★                |
| Emploi               | Très fort       | Fort                 | Fort (entreprise)  |
| Écosystème           | Très vaste      | Vaste                | Intégré            |
| React Native         | ✓               | ✗                    | ✗                  |

## | 🛠 TP 01 — Premier Projet React



- Initialiser
- Créer
- Passer des données
- Composer
- Appliquer



```
# 1. Créer le projet
npm create vite@latest portfolio-react -- --template react
cd portfolio-react

# 2. Installer les dépendances
npm install

# 3. Lancer le serveur de dev
npm run dev
# Ouvrir http://localhost:5173

# 4. Nettoyer le template par défaut
# → Vider App.jsx et App.css
# → Garder uniquement index.css et main.jsx
```



```
// src/components/Header.jsx

// Créer un composant Header avec :
// - Le titre de l'application (prop : title)
// - Un sous-titre (prop : subtitle)
// - Une navigation avec des liens (prop : links = [])

function Header({ title, subtitle, links = [] }) {
  return (
    <header className={styles.header}>
      <div className={styles.brand}>
        <h1>{title}</h1>
        {subtitle && <p>{subtitle}</p>}
      </div>
      <nav>
        {/* TODO : afficher les liens */}
      </nav>
    </header>
  );
}
```





```
// src/components/Card.jsx

// Créer un composant Card affichant une compétence :
// Props : title, description, level (1-5), icon

function Card({ title, description, level = 1, icon = "⚙️" }) {
  const stars = "★".repeat(level) + "☆".repeat(5 - level);

  return (
    <div className="card">
      <span className="card-icon">{icon}</span>
      <h3 className="card-title">{title}</h3>
      <p className="card-description">{description}</p>
      <span className="card-level">{stars}</span>
    </div>
  );
}

export default Card;
```



```
// src/App.jsx
import Header from './components/Header';
import Card from './components/Card';

const skills = [
  { id: 1, title: "HTML/CSS", level: 5, icon: "🌐", description: "Semantic & responsive" },
  { id: 2, title: "JavaScript", level: 4, icon: "⚡", description: "ES6+ & DOM" },
  { id: 3, title: "React", level: 3, icon: "⚙️", description: "En cours d'apprentissage" },
];

function App() {
  return (
    <>
      <Header title="Mon Portfolio" subtitle="Développeur Front-End" />
      <main className="skills-grid">
        {skills.map(skill => (
          <Card key={skill.id} {...skill} />
        ))}
      </main>
    </>
  );
}
```



## Structure de fichiers finale :

```
src/  
├── components/  
│   ├── Header.jsx  
│   ├── Header.module.css  
│   ├── Card.jsx  
│   └── Card.module.css  
├── App.jsx  
├── App.css  
├── index.css  
└── main.jsx
```

## Critères de réussite :



`.map ()`      `key`





## Quiz Jour 1 — Question 1

Quelle affirmation sur `const` est correcte ?

```
const user = { name: "Alice", age: 30 };  
user.age = 31;    // Ligne A  
user = {};        // Ligne B
```

- A) Les deux lignes provoquent une erreur
- B) Seule la ligne A provoque une erreur
- C) Seule la ligne B provoque une erreur ✓
- D) Aucune des deux lignes ne provoque d'erreur

`const` empêche la **réassignation** du binding, pas la mutation du contenu de l'objet.



## Quiz Jour 1 — Question 2

Quelle sortie produit ce code ?

```
const nums = [1, 2, 3, 4, 5];  
const result = nums  
  .filter(n => n % 2 === 0)  
  .map(n => n * 10);  
console.log(result);
```

- A) [1, 3, 5]
- B) [10, 20, 30, 40, 50]
- C) [20, 40] ☒
- D) [2, 4]

`filter(n => n % 2 === 0)` → `[2, 4]` puis `map(n => n * 10)` → `[20, 40]`



## Quiz Jour 1 — Question 3

---

Pourquoi React utilise-t-il un Virtual DOM ?

- A) Pour stocker l'état de l'application
- B) Pour éviter d'écrire du HTML dans JavaScript
- C) Pour minimiser les opérations coûteuses sur le DOM réel ☒
- D) Pour remplacer JavaScript par un autre langage

*Le Virtual DOM permet de **calculer le diff** entre l'ancien et le nouvel état, et n'appliquer au DOM réel que les changements nécessaires.*



## Quiz Jour 1 — Question 4

Lequel de ces JSX est incorrect ?

```
// A
return <><h1>Titre</h1><p>Texte</p></>;

// B
return <div class="container"><p>Texte</p></div>; // ☒ Incorrect

// C
const el = <img src={url} alt="Logo" />;

// D
const list = items.map(i => <li key={i.id}>{i.name}</li>);
```

- **Réponse : B** — En JSX, l'attribut `class` HTML s'écrit `className` ( `class` est un mot réservé JavaScript)



## Quiz Jour 1 — Question 5

---

Quelle est la bonne façon d'initialiser un projet React avec Vite ?

- A) `npx create-react-app mon-app`
- B) `npm create vite@latest mon-app -- --template react` ✓
- C) `npm init react mon-app`
- D) `npx vite create mon-app --react`

*Vite est la solution recommandée en 2025 pour sa vitesse de démarrage et son HMR ultra-rapide.*



# Récapitulatif Jour 1

---

-  **ES6+** : `const` / `let`, arrow functions, destructuration, spread, async/await
-  **Paradigme déclaratif** : décrire *quoi*, pas *comment*
-  **Philosophie React** : composants, unidirectional flow, `UI = f(state)`
-  **Virtual DOM** : diff optimisé → mises à jour minimales du DOM réel
-  **JSX** : syntaxe HTML-like compilée en `React.createElement()`
-  **Vite** : démarrage ultra-rapide, HMR natif, standard industrie
-  **Composants fonctionnels** : fonctions qui reçoivent des props et retournent du JSX

## Pour Aller Plus Loin

---

- **Docs officielles React** : [react.dev](https://react.dev) (nouvelle doc avec hooks)
- **Vite Guide** : [vitejs.dev/guide/](https://vitejs.dev/guide/)
- **ES6+ Reference** : [developer.mozilla.org/fr/docs/Web/JavaScript](https://developer.mozilla.org/fr/docs/Web/JavaScript)
- **JavaScript.info** : [javascript.info](https://javascript.info) (cours ES6+ interactif)
- **React DevTools** : Extension Chrome/Firefox indispensable

*Exercice maison* : Étendez votre portfolio avec une section "Projets" en utilisant un tableau d'objets et `.map()`. Ajoutez un filtrage par technologie.

**Fin du Jour 1** 🎉